Architecture Decision Record

devex-platform — DevEx Intelligence Platform

Date: June 2026 **Author:** Luis Alberto Cruz **Status:** Accepted **Version:** 1.0

Context & Problem Statement

Ten or more independent engineering teams share an AWS account, a GitHub organization, and a CDK-based deployment model — but each team has invented its own hooks, YAML, and CDK patterns. The result: DORA metrics are non-comparable (different definitions of 'deployment'), platform support is bottlenecked (every pipeline change requires platform team involvement), and defect detection is late (failures surface in production before the platform team sees them).

The core tension is **standardization vs. autonomy**. Mandating a single pipeline YAML kills team velocity. Allowing full autonomy makes cross-team measurement impossible. The solution must make the golden path the path of least resistance — teams adopt it not because they are forced to, but because diverging from it costs more than following it.

1. Architecture

The platform separates concerns into two independent layers with a shared contract between them.

The **enforcement layer** operates at developer speed: the `devex` CLI enforces Work ID conventions locally (branch naming, commit format, lint) and `@luicruz01/devex-framework` generates typed GitHub Actions workflows and `LambdaServiceConstruct` CDK abstractions. Teams consume these as versioned packages — they get updates by bumping a dependency, not by copying YAML.

The **analytics layer** operates at platform speed: a versioned `DoraEvent v2` schema (Pydantic + Zod) defines the telemetry contract. A FastAPI **Collector** Lambda validates, enriches with GitHub metadata, and persists events to DynamoDB using a single-table design with GSI patterns for team, repo, stage, and status queries. A pure-Python **Warehouse** engine computes the four DORA metrics. A **DORA Analyst** Lambda calls the Claude API weekly to convert raw metrics into natural-language digests with risk flags and concrete recommendations. A **Streamlit dashboard** surfaces all metrics across teams with elite/high/medium/low ratings.

The **Work ID** is the integration contract threading through every artifact: branch name → commit → PR title → pipeline env var → `DoraEvent.work_id` → DynamoDB → DORA metric → analyst digest. Every deployment is traceable to a business intent and enables lead time computation without instrumenting source control.

Three AI agents augment the ecosystem: **PR Reviewer** (Amazon Q) for automated code review before human reviewers are assigned; **Spec Validator** (AWS Kiro) for validating architecture decisions against `.kiro/steering/` rules before code is written; **DORA Analyst** for converting metric streams into actionable insights — each agent has a defined context boundary and cannot take destructive actions.

2. Homologation — ensuring 10+ teams adopt the ecosystem

The adoption problem is economic, not technical. Teams adopt tools when the cost of non-adoption exceeds the cost of adoption. The design addresses this at four levels:

- **Zero-friction onboarding:** `devex init` generates config, installs git hooks, and detects the stack in under 10 seconds. A new engineer gets the full golden path before their first commit — without reading documentation.
- **Visible cost of divergence:** the DORA dashboard makes adoption measurable. A team that bypasses `devex branch` produces `work_id='N/A'` events — their lead time metric degrades visibly. Compliance is enforced by metrics, not by the platform team.
- **Configurable, not prescriptive:** Work ID pattern, environments, and team name live in `.devex/config.yaml`. A team on Linear changes one regex. No platform involvement required.
- **Context travels with the repo:** `.cursor/rules/` and `.kiro/steering/` files encode platform knowledge into the development environment. Platform mentoring at zero marginal cost.

3. Scalability — avoiding platform team bottleneck

The critical failure mode for platform teams is becoming a review bottleneck. The design prevents this at three levels:

- **Inner-source with proposal gates:** teams propose changes via `[PROPOSAL]` GitHub Issues. Platform team approves the interface contract, not the implementation. Platform reviews take minutes, not days.
- **AI as first-level triage:** Amazon Q filters PRs for convention violations before a human sees them. Kiro rejects non-conformant designs at spec time. DORA Analyst flags regressions before they become support tickets. Empirically, 80% of platform team interruptions are repeatable — this layer eliminates most of them.
- **Schema versioning prevents silent drift:** `DoraEventV2` uses Pydantic `extra='forbid'` and Zod strict parsing. A team that adds an undeclared field gets a validation error at the Collector, not a silent metric corruption six weeks later.

4. Shift-Left Strategy

The cost of a defect compounds with time: a misconfigured CDK construct caught at design time costs minutes; caught in a production incident, it costs hours of MTTR plus a Change Failure Rate spike that takes weeks to recover. The platform implements five detection layers:

- **Design (T+0):** Kiro validates architecture against steering files before any code is written.
- **Commit (T+seconds):** pre-commit runs `devex check` — Work ID in branch, lint, commit format. Cost of fix: edit and re-commit.
- **Push (T+minutes):** pre-push runs the full test suite locally, stack-aware. Failing tests block the push with non-zero exit — no silent pass-through.
- **PR (T+minutes to hours):** PR Pipeline runs tests, schemathesis contract validation, and sandbox deploy. Each stage emits a DoraEvent with real `duration_ms` and `failure_reason`. Amazon Q reviews for security patterns and convention violations.
- **Analytics (T+days):** DORA Analyst detects regressions against p50 baseline and delivers a structured digest weekly. A lead time regression surfaces on Monday morning, not in a quarterly retrospective.

Key Technical Decisions

Decision	Chosen	Rejected alternatives & rationale
Event storage	DynamoDB single-table (GSI2 pattern)	RDS: schema migrations block deploys at scale. Redshift: overkill for <10M events/month, adds cost and ops. DynamoDB gives query
Event ingestion	FastAPI + Mangum Collector Lambda	Kinesis: ordering guarantees not needed at PoC scale. Direct CloudWatch parsing: brittle, hard to validate schema. Collector gives syn
Pipeline generation	TypeScript template literals	github-actions-workflow-ts: adds compile-time safety but reduces readability. Teams inspect and debug generated YAML — readable o
DORA schema	DoraEventV2 Pydantic + Zod	JSON Schema: no runtime validation. Avro/Protobuf: adds serialization complexity. Pydantic + Zod gives runtime validation in both lang
Analytics compute	Python metrics engine + LLM digest	dbt: adds infrastructure dependency. QuickSight: vendor lock-in. Pure Python is testable and deterministic. LLM only for narrative — m

PoC Demonstrated vs Production Gaps (88 passing tests)

Demonstrated (88 passing tests)	Production gaps (known, intentional)
CLI installable via uvx, 3 commands functional	AWS OIDC roles not provisioned — cdk synth validates, not deploys
Framework generates valid, stack-aware GitHub Actions YAML	Amazon Q integration architectural — not wired to live PR webhooks
CDK synth passes with LambdaServiceConstruct on Transactionify stack	Collector not deployed — DynamoDB table not provisioned in AWS
DoraEvent v2: Pydantic extra='forbid' + Zod strict, 14 schema tests	Dashboard reads mock data — Warehouse not connected to live DynamoDB
Collector: schema validation, GitHub env enrichment, GSI persistence	DORA Analyst requires ANTHROPIC_API_KEY provisioned in Lambda env
Warehouse: all 4 DORA metrics with p50/p95, team isolation verified	Go/Clojure lint steps planned — Python and TypeScript implemented
DORA Analyst: Claude API, structured digest, risk flags, dry-run mode	PipelineConfig pre/post hook extension not yet implemented
Streamlit dashboard: Overview, Team Detail, Adoption — live in Docker	